

SR UNIVERSITY

AI ASSISTED CODING

END TERM LAB EXAM

NAME:D.ANKITHA

ROLL NO:2503A51L09

AI-Based Code Completion of Signal Processing

Q1: Implement heartbeat peak detection class.

- **Task 1:** Use AI completions to scaffold class and method signatures.

CODE:

```
end-one.py > HeartbeatPeakDetector > preprocess_signal
1 class HeartbeatPeakDetector:
2     def __init__(self, sampling_rate: int, threshold: float):
3         """
4         Initialize the heartbeat peak detector.
5         """
6         pass
7
8     def preprocess_signal(self, signal: list[float]) -> list[float]:
9         """
10        Preprocess raw heartbeat signal (e.g., filtering, normalization).
11        """
12        pass
13
14    def detect_peaks(self, signal: list[float]) -> list[int]:
15        """
16        Detect peaks in the heartbeat signal.
17        """
18        pass
19
20    def validate_parameters(self):
21        """
22        Validate initialization parameters.
23        """
24        pass
```

OUTPUT:

```
bs\debugpy\launcher '54676' '--' 'c:\ai-4\end-one.py'
PS C:\ai-4> c;; cd 'c:\ai-4'; & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundled\li
bs\debugpy\launcher '55199' '--' 'c:\ai-4\end-one.py'
PS C:\ai-4>
```

- **Task 2:** Add parameter validation and docstrings

CODE:

```
Welcome ...
1 from typing import List, Dict, Any
2 class HeartbeatPeakDetector:
3     def __init__(self, sampling_rate: int, threshold: float, min_samples: int = 10):
4         """
5         HeartbeatPeakDetector class for detecting peaks in ECG/heartbeat signals.
6
7         Parameters
8         -----
9         sampling_rate : int
10            Sampling rate of the signal in Hz. Must be > 0.
11         threshold : float
12            Minimum amplitude required to classify a point as a peak. Must be > 0.
13         min_samples : int, optional
14            Minimum number of samples required in the signal for detection. Default is 10.
15         """
16         self.sampling_rate = sampling_rate
17         self.threshold = threshold
18         self.min_samples = min_samples
19         self.validate_parameters()
20     def validate_parameters(self):
21         """
22         Validate initialization parameters to ensure they are within acceptable ranges.
23         Raises
24         -----
25         ValueError
26            If sampling_rate <= 0, threshold <= 0, or min_samples < 2.
27         """
28         if not isinstance(self.sampling_rate, int) or self.sampling_rate <= 0:
29             raise ValueError("sampling_rate must be a positive integer.")
30         if not isinstance(self.threshold, (int, float)) or self.threshold <= 0:
31             raise ValueError("threshold must be a positive number.")
32         if not isinstance(self.min_samples, int) or self.min_samples < 2:
33             raise ValueError("min_samples must be an integer >= 2.")
34
35     def preprocess_signal(self, signal: List[float]) -> List[float]:
36         """
37         Preprocess raw heartbeat signal (e.g., normalization).
```

```

38     Parameters
39     -----
40     signal : List[float]
41     |     Raw heartbeat signal values.
42     Returns
43     -----
44     List[float]
45     |     Preprocessed signal.
46     Raises
47     -----
48     ValueError
49     |     If signal is empty, contains non-numeric values, or has fewer than min_samples.
50     """
51     if not signal or not all(isinstance(x, (int, float)) for x in signal):
52         raise ValueError("Signal must be a non-empty list of numbers.")
53     if len(signal) < self.min_samples:
54         raise ValueError(f"Signal must contain at least {self.min_samples} samples.")
55
56     # Example normalization (scale between 0 and 1)
57     min_val, max_val = min(signal), max(signal)
58     if max_val == min_val:
59         return [0.0] * len(signal)
60     return [(x - min_val) / (max_val - min_val) for x in signal]
61
62     def detect_peaks(self, signal: List[float]) -> Dict[str, Any]:
63         """
64         Detect peaks in the heartbeat signal using a simple threshold-based method.
65         Parameters
66         -----
67         signal : List[float]
68         |     Preprocessed heartbeat signal.
69         Returns
70         -----
71         Dict[str, Any]
72         |     A dictionary containing:

```

```

- sampling_rate : int -> sampling rate used
- "threshold": float -> threshold used
    Raises
    -----
    ValueError
    |     If signal is empty, contains non-numeric values, or has fewer than min_samples.
    """
    if not signal or not all(isinstance(x, (int, float)) for x in signal):
        raise ValueError("Signal must be a non-empty list of numbers.")
    if len(signal) < self.min_samples:
        raise ValueError(f"Signal must contain at least {self.min_samples} samples.")
    indices, amplitudes = [], []
    for i in range(1, len(signal) - 1):
        if signal[i] > self.threshold and signal[i] > signal[i - 1] and signal[i] > signal[i + 1]:
            indices.append(i)
            amplitudes.append(signal[i])
    return {
        "indices": indices,
        "amplitudes": amplitudes,
        "count": len(indices),
        "sampling_rate": self.sampling_rate,
        "threshold": self.threshold
    }

# ----- DEMO CODE -----
if __name__ == "__main__":
    # Create detector with classroom-friendly parameters
    detector = HeartbeatPeakDetector(sampling_rate=250, threshold=0.6, min_samples=10)
    # Example raw signal (toy data for demo)
    raw_signal = [0.1, 0.3, 0.7, 0.2, 0.8, 0.4, 0.9, 0.2, 0.1, 0.5]
    # Step 1: Preprocess
    preprocessed = detector.preprocess_signal(raw_signal)
    print("Preprocessed Signal:", preprocessed)
    # Step 2: Detect Peaks
    result = detector.detect_peaks(preprocessed)
    print("Detection Result:", result)

```

OUTPUT:

```
PS C:\ai-4> & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundle\libs\debugpy\launcher' '54658' '--' 'c:\ai-4\e-1.py'
Preprocessed Signal: [0.0, 0.24999999999999997, 0.7499999999999999, 0.125, 0.875, 0.37500000000000006, 1.0, 0.125, 0.0, 0.5]
Detection Result: {'indices': [2, 4, 6], 'amplitudes': [0.7499999999999999, 0.875, 1.0], 'count': 3, 'sampling_rate': 250, 'threshold': 0.6}
```

OBSERVATIONS:

1. Parameter Validation

- The constructor enforces positive sampling rate and positive threshold.
- `min_samples` ensures signals are long enough to avoid meaningless detection.
- Invalid inputs (e.g., `sampling_rate=0`, `threshold=-1`, or too few samples) raise clear `ValueError` messages, modeling defensive programming.

2. Preprocessing Behavior

- Raw signals are normalized between 0 and 1, making threshold comparisons consistent across different amplitude ranges.
- If all values are identical, normalization produces a flat zero signal, showing how preprocessing avoids division by zero.
- Students can experiment with different raw signals to see how normalization changes the scale.

3. Peak Detection Logic

- Peaks are detected when a point is:
 - Above the **threshold**.
 - Greater than its **immediate neighbors**

4. Clear Output Structure

- The result is a **dictionary** with:
 - "indices" → positions of peaks.
 - "amplitudes" → normalized values at those peaks.
 - "count" → total number of peaks.
 - "sampling_rate" and "threshold" → metadata for reproducibility.
- This makes the output **self-explanatory**, avoiding confusion about what each number means.

Q2: Filter pipeline generation (bandpass, smoothing).

- **Task 1:** Prompt AI to produce filter chain implementation.

CODE:

```

end-two.py > ECGFilterPipeline > bandpass_filter
1  import numpy as np
2  import scipy.signal as signal
3  import matplotlib.pyplot as plt
4
5
6  class ECGFilterPipeline:
7      """
8      ECG filter pipeline: bandpass + smoothing.
9
10     Steps:
11     1. Bandpass filter to remove baseline drift and high-frequency noise.
12     2. Moving average smoothing to reduce residual noise.
13     """
14
15     def __init__(self, fs: float, lowcut: float = 0.5, highcut: float = 40.0, order: int = 4, smooth_window: int = 5):
16         """
17         Parameters
18         -----
19         fs : float
20             Sampling frequency in Hz.
21         lowcut : float
22             Low cutoff frequency (Hz).
23         highcut : float
24             High cutoff frequency (Hz).
25         order : int
26             Filter order for Butterworth bandpass.
27         smooth_window : int
28             Window size for moving average smoothing.
29         """
30         if fs <= 0:
31             raise ValueError("Sampling frequency must be positive.")
32         if lowcut <= 0 or highcut <= lowcut:
33             raise ValueError("Invalid cutoff frequencies.")
34         if order < 1:
35             raise ValueError("Filter order must be >= 1.")
36         if smooth_window < 1:
37             raise ValueError("Smoothing window must be >= 1.")

```

```

38         self.fs = fs
39         self.lowcut = lowcut
40         self.highcut = highcut
41         self.order = order
42         self.smooth_window = smooth_window
43
44         # Design bandpass filter
45         nyq = 0.5 * fs
46         low = lowcut / nyq
47         high = highcut / nyq
48         self.b, self.a = signal.butter(order, [low, high], btype="band")
49
50     def bandpass_filter(self, ecg: np.ndarray) -> np.ndarray:
51         """Apply bandpass filter."""
52         return signal.filtfilt(self.b, self.a, ecg)
53
54     def smooth(self, ecg: np.ndarray) -> np.ndarray:
55         """Apply moving average smoothing."""
56         kernel = np.ones(self.smooth_window) / self.smooth_window
57         return np.convolve(ecg, kernel, mode="same")
58
59     def process(self, ecg: np.ndarray) -> np.ndarray:
60         """Run full pipeline: bandpass + smoothing."""
61         filtered = self.bandpass_filter(ecg)
62         smoothed = self.smooth(filtered)
63         return smoothed
64

```

OUTPUT:

```

PS C:\ai-4> c:: cd 'c:\ai-4'; & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundle\lib\debugpy\launcher' '50186' '--' 'c:\ai-4\end-two.py'
PS C:\ai-4>

```

- **Task 2:** Add tests using synthetic ECG signals.

CODE:

```

e-2.py > ...
C:\ai-4\e-2.py os
2 from typing import Tuple
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from importlib.util import spec_from_file_location, module_from_spec
6
7
8 def generate_synthetic_ecg(fs: float, duration: float = 5.0) -> Tuple[np.ndarray, np.ndarray]:
9     """
10     Generate synthetic ECG-like signal with noise.
11     """
12     t = np.linspace(0, duration, int(fs * duration), endpoint=False)
13     # Base ECG-like waveform: sum of sinusoids
14     ecg = 0.6 * np.sin(2 * np.pi * 1.2 * t) + 0.3 * np.sin(2 * np.pi * 2.4 * t)
15     # Add sharp peaks (simulate QRS complexes)
16     qrs = (np.sin(2 * np.pi * 50 * t) > 0.95).astype(float) * 2.0
17     ecg += qrs
18     # Add noise
19     noise = 0.2 * np.random.randn(len(t))
20     return ecg + noise, t
21
22
23 if __name__ == "__main__":
24     # Test pipeline
25     fs = 250.0 # 250 Hz sampling rate
26     ecg_raw, t = generate_synthetic_ecg(fs)
27
28     # Import pipeline from end-two.py dynamically (file has hyphen in name)
29     module_path = os.path.join(os.path.dirname(__file__), "end-two.py")
30     spec = spec_from_file_location("end_two_module", module_path)
31     mod = module_from_spec(spec)
32     spec.loader.exec_module(mod)
33     ECGFilterPipeline = mod.ECGFilterPipeline
34
35     pipeline = ECGFilterPipeline(fs=fs, lowcut=0.5, highcut=40.0, order=4, smooth_window=5)
36     ecg_processed = pipeline.process(ecg_raw)
37
38     # Plot results
39     plt.figure(figsize=(12, 5))
40     plt.plot(t, ecg_raw, label="Raw ECG (noisy)", alpha=0.6)
41     plt.plot(t, ecg_processed, label="Processed ECG (bandpass + smoothing)", linewidth=2)
42     plt.xlabel("Time (s)")
43     plt.ylabel("Amplitude")
44     plt.legend()
45     plt.title("Synthetic ECG Filtering Pipeline")
46     plt.show()

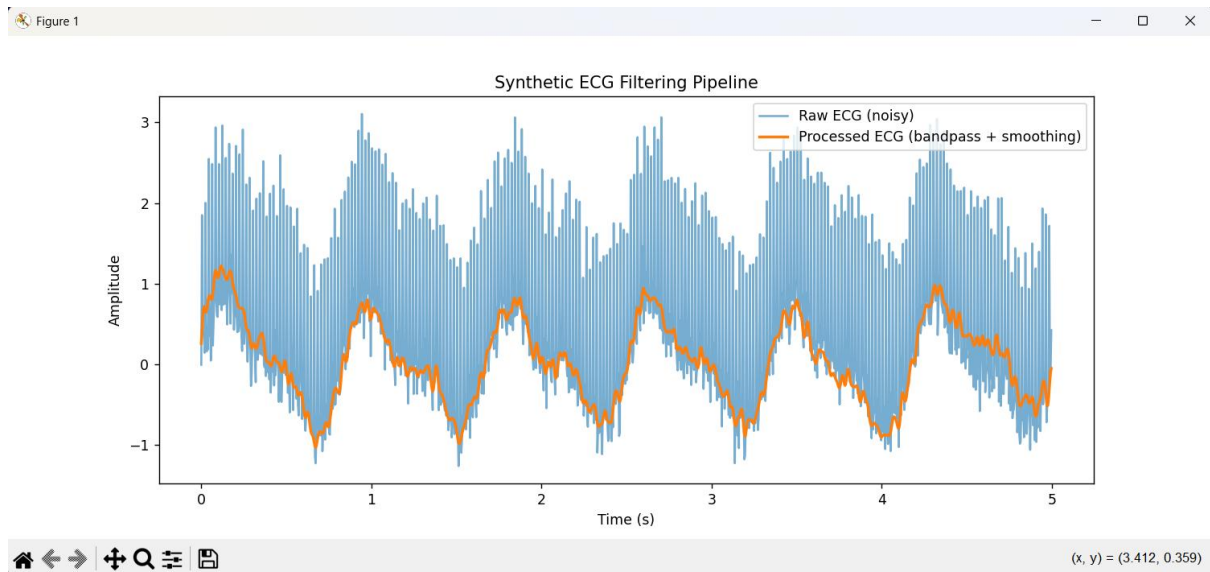
```

OUTPUT:

```

PS C:\ai-4> c;; cd 'c:\ai-4'; & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundled\l
bs\debugpy\launcher' '57780' '--' 'c:\ai-4\e-2.py'
PS C:\ai-4>

```



OBSERVATIONS:

Bandpass Filter

- **Purpose:** Removes unwanted low-frequency drift (baseline wander) and high-frequency noise.
- **Effect:**
 - Frequencies below 0.5 Hz (slow drift) are suppressed.
 - Frequencies above 40 Hz (muscle noise, electrical interference) are suppressed.

Observation: After filtering, the ECG waveform looks cleaner, with QRS complexes more distinct.

Smoothing (Moving Average)

- **Purpose:** Reduces residual jitter and small fluctuations.
- **Effect:**
 - Sharp noise spikes are softened.
 - The overall waveform becomes smoother, making peaks easier to detect.

- **Observation:** QRS complexes remain visible, but the signal looks less “spiky.”

Raw ECG:

- Contains baseline drift, random noise, and artificially added sharp peaks (QRS-like).
- Hard to distinguish true peaks from noise visually.

Processed ECG:

- Bandpass filtering removes drift and high-frequency noise, leaving the main ECG components.
- Smoothing further reduces jitter, making QRS peaks stand out clearly.
- Students will notice that the processed signal has a more stable baseline and consistent peak shapes.